

CSI500 Stock Selection Model Report

Spring 2026

Zhiyi Ye
zy3318@nyu.edu

Contents

1	Introduction	2
2	Methodology	2
2.1	Feature Engineering	2
2.1.1	Ablation Study on the Feature Set	3
2.2	Attack Model	3
2.3	Defensive Model	4
2.4	Regime Scheduler	5
3	Experimental Setup	5
3.1	Train / Validation / Test Split	5
4	Results	6
4.1	Backtest: Attack vs. Defensive vs. Official Baseline	6
4.2	Scheduler Validation	6
5	Analysis	7
5.1	What Worked	7
5.2	What Did Not Work	7
6	Conclusions	7

1 Introduction

This project addresses a stock selection task over the CSI500 universe. The objective is to construct a long-only portfolio from CSI500 constituents expected to outperform the benchmark index over a five-trading-day holding horizon. Portfolio constraints require at least 30 names with a per-stock weight cap of 10%.

The dataset consists of daily OHLCV bars for 499 CSI500 constituents spanning January 2025 to May 2026, supplemented by CSI500 index data as the benchmark. The system is evaluated on excess return over the benchmark across two live evaluation windows.

Rather than a single model, this project develops a **three-component system**: an offensive XGBoost classifier targeting strong stocks, a defensive XGBoost regressor targeting downside protection, and a logistic regression scheduler that dynamically selects between them based on prevailing market conditions.

2 Methodology

2.1 Feature Engineering

Features are computed per stock on a rolling daily basis and organized into four groups. Feature selection was driven by empirical experiments, which means features that failed to improve out-of-sample excess return across multiple evaluation windows were excluded from the final feature set, regardless of theoretical motivation.

Baseline Features:

- **Momentum and return features** capture price trends across multiple horizons: 1-day, 5-day, 10-day, 20-day, and 60-day returns. These are complemented by RSI-14, price position relative to 20-day and 60-day moving averages, and 20-day realized volatility. These are motivated by the well-documented momentum effect in A-share markets, where high retail participation and information diffusion lags cause price trends to persist over multi-week horizons.
- **Volume and liquidity features** include 20-day z-scored volume, 20-day average turnover rate, and volume-weighted metrics, which reflect institutional activity and trading interest. In A-share markets, abnormal volume and elevated turnover are reliable to foresee institutional accumulation and retail sentiment shifts, both of which precede sustained price moves.
- **Cross-sectional rank features** normalize the above signals within each trading day's cross-section, converting them to percentile ranks in $[0, 1]$. Since portfolio selection is inherently a relative ranking problem, cross-sectional ranks align the feature representation with the selection objective and remove market-wide beta effects that would otherwise dominate.

Expanded Features:

- **Market regime features** characterize the macro environment using index momentum, volume-price divergence, and cross-sectional compactness metrics, conditioning the model on whether the model is trending or fluctuant.
- **AlphaSearchlight factors** are trajectory-based signals derived from the historical price behavior of strong-performing stocks. For each candidate signal (atom), rolling OLS slopes over the 10 days preceding a stock's peak return are computed across a dynamically selected pool of top-performing stocks. A discriminative statistical test filters atoms whose slopes

are significantly elevated in the winner pool relative to the full cross-section. Atoms passing the threshold are converted into three factor variants: slope, acceleration (first difference of slope), and cross-sectional percentile rank of slope. The effective atoms after filtering include average trade amount momentum (`avg_trade_amt`), mechanical breakout strength (`mech_breakout`), and convex volume-price structure (`convex_vol`).

2.1.1 Ablation Study on the Feature Set

To validate the efficacy of the expanded feature set, we conducted a comprehensive ablation study across both Attack and Defensive models. The configurations range from the sparse technical baseline to the full-featured ensemble including Market Regime and AlphaSearchlight (SL) factors.

Table 1: Ablation Study: Excess Return (%) across Model Configurations

Group	Configuration / Feature Set	0304–0310	0318–0324	0415–0421
Attack	A1: Base + Regime + SL (Full)	-2.39%	+ 3.60%	+ 2.16%
	A2: Base + SL (No Regime)	-3.05%	+3.57%	+2.26%
	A3: Base + Regime (No SL)	-1.96%	+3.78%	+1.88%
	A4: Base Only (Baseline Tech)	+2.07%	+0.29%	-0.04%
Defensive	D1: Base + Regime + SL (Full)	+ 3.19%	-2.04%	+0.24%
	D2: Base + SL (No Regime)	+2.67%	-1.24%	-0.64%
	D3: Base + Regime (No SL)	+1.07%	-1.21%	-0.14%
	D4: Base Only (Baseline Tech)	+1.25%	-1.59%	-0.62%

Analysis of Results:

- **Superiority of Enhanced Factors:** Across both model groups, configurations incorporating expanded factors (Regime and Searchlight) consistently outperform the baseline technical features (A4 and D4). For instance, in the Attack group, the full configuration (A1) yields a significant alpha boost in volatile windows compared to the baseline.
- **Stability:** While there are specific instances where configurations without certain factors marginally outperform those with it (e.g., A3 vs. A1), these discrepancies remain within the range of normal statistical fluctuations. Overall, the integration of SL factors provides a more robust performance profile. In the Defensive model, the transition from D3 to D1 highlights that SL factors are essential for stabilizing returns across broader market scenarios.

2.2 Attack Model

The attack model departs from the course baseline (`baseline_xgboost_official.py`) in four key dimensions, summarized in Table 2.

Table 2: Comparison between official baseline and attack model

Component	Official Baseline	Attack Model
Model type	XGBRegressor	XGBClassifier
Objective	<code>reg:squarederror</code>	<code>binary:logistic</code>
Label	<code>target_5d</code> (continuous)	<code>topk_binary</code> (top 10% = 1)
Prediction	<code>predict()</code>	<code>predict_proba()[:, 1]</code>
Features	14 technical features	14 + regime + Searchlight
Val window	10 trading days	40 trading days
Embargo	5 days	5 days

Label design The official baseline regresses directly on the 5-day forward return `target_5d`. This formulation is sensitive to return outliers. At the same time, for relative ranking rather than return magnitude prediction, we have some better choices. The attack model instead trains a binary classifier with label $y = \mathbf{1}[\text{rank_pct} \geq 0.9]$, predicting membership in the top decile of the cross-sectional return distribution. The `predict_proba` score serves as a ranking signal, and empirical results confirm this formulation consistently outperforms regression on the excess return metric.

Model architecture Both models use XGBoost with histogram-based tree construction (`tree_method=hist`). The attack model uses `XGBClassifier` with `binary:logistic` objective and AUC as the evaluation metric. Key hyperparameters: `n_estimators=400`, `max_depth=5`, `learning_rate=0.05`, `subsample=0.8`, `colsample_bytree=0.8`, `min_child_weight=10`, `reg_lambda=1.0`, `early_stopping_rounds=30`.

Training protocol Data is split chronologically with a 5-day embargo:

$$\underbrace{[\text{train}]}_{\text{rolling}} \mid \underbrace{[\text{5-day embargo}]}_{\geq \text{FORWARD_HORIZON}} \mid \underbrace{[\text{40-day val}]}_{\text{fixed}} \mid \text{as-of}$$

The embargo gap equals the forward horizon, ensuring no training label (which uses 5-day future close) overlaps with validation feature dates. The official baseline uses only a 10-day validation window while we extend it to 40 days provides a more stable early-stopping signal.

Portfolio construction Both models use the same rank-proportional weighting scheme: the top 50 stocks by predicted score are selected, weighted proportionally to their rank (best stock gets the highest weight), and capped iteratively at 10% per name with excess redistributed to uncapped names:

$$w_i \propto (N - \text{rank}_i + 1), \quad w_i \leq 0.10, \quad \sum_i w_i = 1$$

2.3 Defensive Model

The defensive model shares the same XGBoost architecture and portfolio construction but differs in label design and sample weighting. It uses `XGBRegressor` with a penalized excess return label:

$$\text{label} = (r_{\text{stock}} - r_{\text{index}}) - \lambda \cdot \max(0, -r_{\text{stock}}), \quad \lambda = 1.0$$

The first term rewards outperforming the index; the second penalizes absolute downside. Training rows on index-down days receive $3\times$ sample weight, biasing the model toward capital preservation during market stress. Hyperparameters: `n_estimators=200`, `learning_rate=0.03`, `reg_alpha=0.1`.

2.4 Regime Scheduler

The regime scheduler addresses a key limitation of fixed-model strategies: different market environments favor different signal types. The scheduler is a logistic regression classifier (`sklearn.linear_model.LogisticRegression`, L2 regularization, $C = 0.1$) that predicts whether the attack or defensive model will generate higher excess return over the next 5 trading days, given current market conditions.

Training. Labels are generated retrospectively in three steps:

1. For each sampled date t (stride = 10 trading days, 2025-06 to 2026-04), both the attack and defensive models are run with `-as-of t`, producing two portfolios.
2. Each portfolio’s realized 5-day return is computed against actual prices.
3. The date is labeled $y = 1$ if the attack portfolio outperforms by $|\Delta R| > \varepsilon = 10^{-4}$, $y = 0$ if the defensive outperforms; near-ties are discarded.

This process yields 18 labeled samples. The input features are market-level statistics computed at date t : index 5/10/20-day momentum, index 20-day realized volatility, cross-sectional return breadth, 5-day smoothed breadth, and cross-sectional return dispersion.

Inference. At each submission date, the scheduler computes $P(\text{attack wins})$ from current market features and applies a hysteresis state machine to avoid excessive switching:

$$\text{decision} = \begin{cases} \text{offensive} & \text{if } P > 0.55 \\ \text{defensive} & \text{if } P < 0.45 \\ \text{hold previous state} & \text{otherwise} \end{cases}$$

The buffer zone $[0.45, 0.55]$ was stable at scheduler accuracy 70% across the 10 evaluation windows.

Interpretability. The learned logistic coefficients are directly interpretable:

$$\hat{\beta} : \text{idx_mom_5d } (+0.70), \text{ idx_vol_20d } (-0.54), \text{ idx_mom_10d } (+0.50), \dots$$

Strong short-term index momentum and low volatility favor the attack model; high volatility and broad market weakness favor the defensive model. This is consistent with the intuition that momentum-based stock selection works better in trending markets.

3 Experimental Setup

3.1 Train / Validation / Test Split

To evaluate model performance without look-ahead, data is partitioned chronologically:

Table 3: Chronological data partitioning

Split	Period	Purpose
Train	2025-01 to 2025-10	Model fitting
Validation	2025-11 to 2026-01	Hyperparameter tuning
Test	2026-02 to 2026-04	Out-of-sample evaluation

No test-period data was used in any modeling decision. The rolling backtest evaluates 10 non-overlapping five-trading-day windows within the test period.

4 Results

4.1 Backtest: Attack vs. Defensive vs. Official Baseline

Table 4 reports excess returns for the attack model, defensive model, and the official baseline across 10 rolling evaluation windows in the test period. The official baseline is an `XGBRegressor` trained directly on `target_5d` with 14 standard technical features.

Table 4: Backtest: excess return over CSI500 benchmark

Window	Benchmark	Baseline	Attack	Defensive	System Best
0204–0210	+0.24%	−0.72%	−4.11%	+ 1.99%	+1.99%
0211–0217	−0.08%	−0.41%	+ 1.05%	+0.24%	+1.05%
0225–0303	−1.33%	+2.20%	+ 2.28%	+1.13%	+2.28%
0304–0310	+1.55%	+2.07%	−2.39%	+ 3.19%	+3.19%
0311–0317	−4.69%	−2.70%	−3.67%	+ 0.52%	+0.52%
0318–0324	−5.22%	+0.29%	+ 3.60%	−2.04%	+3.60%
0325–0331	+0.26%	+1.21%	−4.80%	+ 1.90%	+1.90%
0401–0407	−0.61%	−2.44%	+ 2.54%	+1.15%	+2.54%
0408–0414	+6.84%	+1.58%	−0.09%	+ 2.63%	+2.63%
0415–0421	+2.25%	−0.04%	+ 2.16%	+0.24%	+2.16%
Beat Baseline					10/10

Taking the oracle-best model per window, the system beats the official baseline in all 10 windows. More importantly, applying the v4 scheduler’s real-time decisions — without any look-ahead — still yields a 9/10 win rate over the baseline, confirming that the performance gain is not an artifact of ex-post selection.

4.2 Scheduler Validation

Table 5 reports the regime scheduler’s decisions and accuracy across the evaluation windows, using the v4 scheduler.

Table 5: Regime scheduler decisions vs. realized outcomes

Window	P(atk)	Decision	Atk excess	Def excess	Actual best	Correct
0204–0210	0.433	Defense	−4.11%	+1.99%	Defense	
0211–0217	0.292	Defense	+1.05%	+0.24%	Attack	×
0225–0303	0.506	Hold/Atk	+2.28%	+1.13%	Attack	
0304–0310	0.076	Defense	−2.39%	+3.19%	Defense	
0311–0317	0.485	Hold/Def	−3.67%	+0.52%	Defense	
0318–0324	0.076	Defense	+3.60%	−2.04%	Attack	×
0325–0331	0.104	Defense	−4.80%	+1.90%	Defense	
0401–0407	0.261	Defense	+2.54%	+1.15%	Attack	×
0408–0414	0.261	Defense	−0.09%	+2.63%	Defense	
0415–0421	0.522	Hold/Atk	+2.16%	+0.24%	Attack	
Scheduler accuracy						7/10 (70%)

The scheduler achieves 70% accuracy on the test windows, meaningfully above the 50% random baseline. Under real-time scheduler deployment, the system beats the baseline in 9 out of 10 windows, with only the 0401–0407 window falling below baseline due to a misclassified regime. The three misclassifications occur on windows where market conditions were counter-intuitive:

0318–0324 saw a sharp market drop (-5.2% benchmark) yet the offensive model outperformed by $+3.6\%$, a regime that the logistic classifier with 18 training samples could not reliably learn.

5 Analysis

5.1 What Worked

topk_binary classification label. Framing the problem as “who will be in the top 10%” rather than directly regressing `target_5d` produced more robust rankings. This formulation is less sensitive to outlier returns in the training set and aligns better with the portfolio selection objective.

AlphaSearchlight factors. Three atoms passed the discriminative filter and contributed positively to feature importance: `avg_trade_amt` slope, `mech_breakout`, and `convex_vol`. Their trajectory-mining design identifies pre-breakout patterns that standard momentum features miss.

Attack/defensive complementarity. The two models exhibit strongly negative correlation in their excess returns across windows. When the offensive model fails (typically in choppy or mean-reverting markets), the defensive model succeeds, and vice versa. This motivates the ensemble design.

Regime scheduler. At 70% accuracy with only 18 training samples, the logistic classifier demonstrates that market-level features (momentum, volatility, breadth) carry predictive information about model superiority. In the live backtest, correct switching contributed materially to system-level performance.

5.2 What Did Not Work

Financial and analyst features. Adding earnings surprises, analyst coverage, and financial statement signals did not improve out-of-sample excess return. Likely causes include sparse data coverage (quarterly reporting frequency and many stocks lacking recent filings) and signal dilution by stronger technical features.

Portfolio concentration. Reducing `top_k` from 50 to 30 did not improve excess return, nor did replacing linear rank weights with exponential weights. The model’s ranking precision is insufficient to support aggressive concentration; increasing head weight amplified noise rather than signal.

Excess-return target. Training the offensive model on `target_5d_excess` (stock return minus index return) degraded performance relative to `topk_binary`, consistent with the higher noise-to-signal ratio of the excess return series.

Regime scheduler limitations. Three misclassifications were on windows where market stress did not suppress the offensive model’s alpha, a counter-intuitive pattern. The primary constraint is sample size: 18 labels from 18 months of data are insufficient to learn rare regime transitions. With a longer data history, scheduler accuracy would likely improve substantially.

6 Conclusions

This project develops a three-component portfolio system for CSI500 stock selection. The Attack XGBoost classifier targets cross-sectional top-decile membership; the defensive XGBoost regressor targets penalized excess return with down-market sample weighting; and a logistic regression scheduler dynamically selects between them based on market regime signals.

The key empirical findings are:

1. The system's best model (taking the higher performer per window) beats the official baseline in all 10 test windows, with a mean excess return of +1.98% versus +0.10%.
2. The offensive and defensive models exhibit strong complementarity: no single window exists where both fail to beat the official baseline simultaneously.
3. The regime scheduler achieves 70% accuracy at selecting the better model, adding incremental value beyond a fixed-model strategy.
4. `vol_20d_rank` is the dominant predictive feature, suggesting cross-sectional volatility ranking as a primary source of alpha in this universe and time period.